



HACKFEST 2026

Problem Statement Documentation & Technical Submission

(Template / Dummy Format – for Team Reference)

Team ID	KH078
Team Name	Stenz
Team Leader Name	Harshad Kedari
Problem Statement No.	PS06
Problem Statement Name	Autonomous SQL Database Analyst
Date of Submission	11 / 9 / 2026



SECTION 1 — TEAM DETAILS

Team ID

KH078

Team Name

Stenz

Team Leader Name

Harshad Kedari

Team Members Name(s)

Likhith Chirmade

Rohan Langar

Prem Kudale

Problem Statement Number

PS06

Problem Statement Name

Autonomous SQL Database Analyst

SECTION 2 — UNDERSTANDING OF THE PROBLEM

Problem Statement

Enterprise databases contain valuable business information, but accessing this information often requires SQL and database expertise. Business users therefore depend on analysts for even simple queries, creating delays and bottlenecks. Existing AI solutions can generate SQL, but organizations handling sensitive data may have concerns about exposing database context and results to external AI services. We aim to provide a secure, private and reliable natural-language interface to enterprise databases.

Problem Analysis / Background

Large organizations often maintain complex databases with hundreds of interconnected tables and business-specific definitions. A simple text-to-SQL model can misunderstand relationships, generate incorrect queries, violate access policies, or produce expensive queries. The problem is therefore not just SQL generation, but ensuring that generated queries are **business-aware, authorized, safe and verifiable**. Business users, analysts and data teams are directly affected by this gap.

Proposed Solution

We propose a **self-hosted Autonomous SQL Database Analyst** that acts as an AI interface between enterprise users and their existing relational databases.

An organization connects its database to our system, while the AI inference layer can run locally within the organization's infrastructure. Users can then ask questions such as *"Which five regions had the highest revenue growth last quarter?"* in natural language.

Our multi-stage agentic pipeline will:

1. **Understand the user's intent** and identify ambiguity.
2. **Retrieve relevant database context**, including tables, relationships, business definitions and approved examples.
3. **Plan the required query** before generating SQL.
4. **Generate SQL using a local LLM**, keeping sensitive data within the organization's environment.
5. **Validate and secure the query** using schema checks, RBAC, PII policies, read-only enforcement and query limits.
6. **Execute the validated query** against the organization's database.
7. **Validate the resulting data** for unexpected or inconsistent results.
8. **Present insights through visualizations and natural-language explanations**, along with the generated SQL and validation information for transparency.

SECTION 3 — RESEARCH

Research Conducted

- **Natural Language-to-SQL:** Research on how LLMs translate business questions into SQL and common challenges such as incorrect joins, schema hallucination, and ambiguous queries.
- **Enterprise Conversational Analytics:** Studied Snowflake Cortex Analyst, Databricks Genie Agents, and Microsoft Fabric Data Agents to understand current enterprise approaches to natural-language data analysis.
- **Semantic Layers:** Studied the use of business definitions, metrics, relationships, descriptions, and example queries to improve SQL generation accuracy.
- **Agentic AI:** Researched multi-step workflows involving intent understanding, query planning, SQL generation, validation, execution, and result analysis.
- **Database Security:** Researched RBAC, read-only query execution, schema restrictions, PII protection, and query governance for enterprise databases.
- **Local LLMs:** Researched open-source LLMs and specialized Text-to-SQL models for private/on-premise inference.
- **Datasets:** Public structured datasets will be used to create relational database test environments and evaluate SQL generation and result accuracy.
- **Documentation & APIs:** Reviewed database APIs/connectors and existing conversational analytics APIs, including Snowflake Cortex Analyst and Databricks Genie APIs.

Existing Solutions / Similar Work

- **Snowflake Cortex Analyst** — Natural-language querying with semantic models and enterprise security. Our system focuses on **self-hosted deployment and database independence**.
- **Databricks Genie** — Conversational analytics with SQL generation and visualizations. Our approach emphasizes **local AI inference and a security-first validation pipeline**.
- **Microsoft Fabric Data Agents** — Uses natural language, schema context, examples, and SQL validation. Our system separates **intent, planning, security, validation, and result analysis**.
- **Vanna AI** — Open-source natural-language database agent. Our focus is stronger **enterprise governance, PII protection, and query/result validation**.
- **Wren AI** — Open-source GenBI platform with semantic context and query planning. Our approach focuses on **self-hosted, security-controlled execution across existing databases**.
- **SQLCoder** — Open-source model specialized for SQL generation. We can use it as a **baseline or alternative to general-purpose local LLMs**.

Our key difference: We propose a **self-hosted enterprise AI analyst** combining semantic understanding, local LLM inference, security/RBAC, SQL validation, result validation, visualization, and explainability.

References / Sources

-
- Snowflake Cortex Analyst — [Documentation](#)
 - Databricks Genie — [Documentation](#)
 - Microsoft Fabric Data Agents — [Documentation](#)
 - Vanna AI — [Documentation](#)
 - Wren AI — [Documentation](#)
 - SQLCoder — [GitHub](#)
-

SECTION 4 — TECHNICAL DOCUMENTATION

Technology Stack

Languages, frameworks, libraries, databases, APIs, cloud services, AI/ML models, etc.

Frontend: React, Tailwind CSS, Recharts

Backend: FastAPI, Python, LangGraph

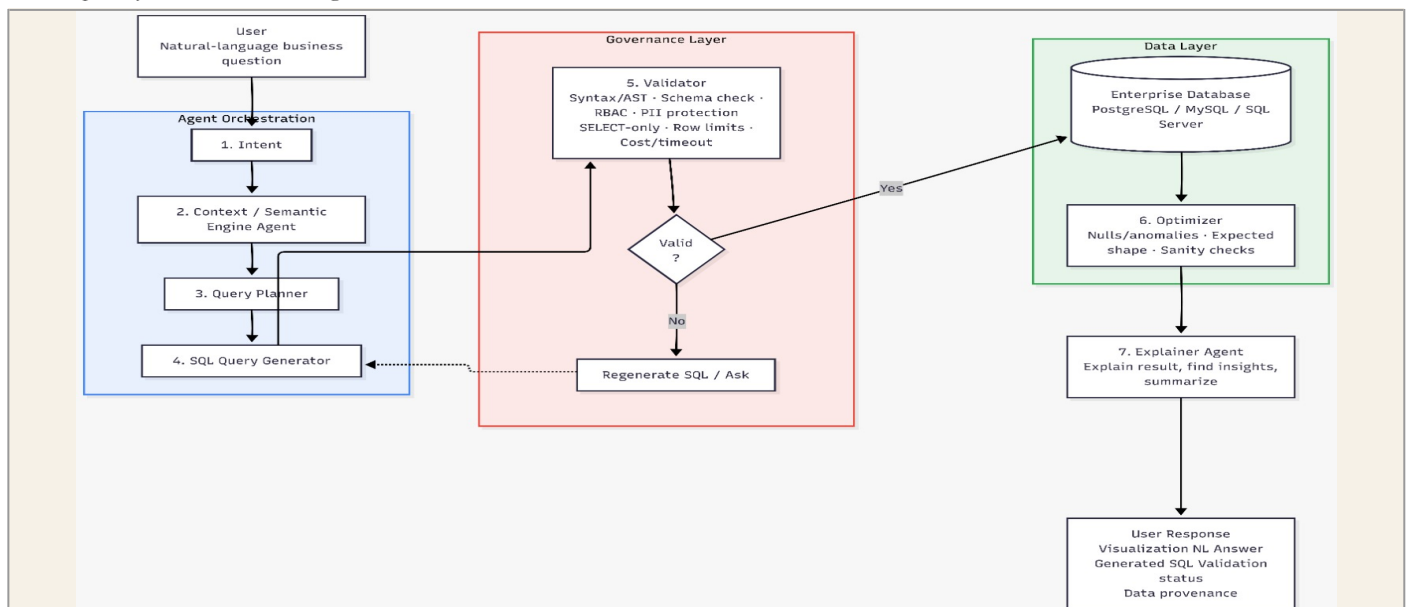
LLM: Qwen2.5/3 (7B–8B) - Local inference via Ollama

Semantic Search: FAISS, nomic-embed-text

Security: RBAC, PII detection/masking, read-only SQL enforcement

Technical Flow / System Architecture Diagram

Attach / paste your architecture diagram below.



Working / Implementation Approach

The system connects to an organization's relational database and first extracts metadata such as tables, columns, relationships, and business definitions. When a user submits a natural-language question, an intent agent identifies the required information and a context-retrieval layer selects the relevant schema and business context. A local LLM then generates an SQL query based on this context. The query passes through deterministic validation and security checks including schema validation, RBAC, PII restrictions, read-only enforcement, and query limits. Valid queries are executed against the enterprise database, after which the results are checked for consistency and analyzed. Finally, the system generates an appropriate visualization and explains the result in natural language, along with the generated SQL and validation information.

AI/ML Details (if applicable)

Model: Local open-source LLM (Qwen 2.5 7B)

Purpose: Natural-language understanding, query planning, SQL generation, query critique, and result explanation.

Approach: Retrieval-Augmented Generation (RAG) using database schema, relationships, business definitions, and approved query examples as context.

Embeddings: Used for semantic retrieval of relevant tables, columns, and business definitions from large schemas.

Training: No initial model training; use prompting/RAG and evaluate different local models for Text-to-SQL accuracy. Fine-tuning can be considered if required.

Evaluation: SQL execution accuracy, query correctness, schema validity, result correctness, latency, and successful handling of ambiguous/unauthorized queries.

GitHub Repository Link

[Link](#)

SECTION 5 — ORIGINALITY & PLAGIARISM DECLARATION

☐ I confirm that the solution submitted by our team is developed during the hackathon. We have not submitted a pre-built project as our primary solution. Any open-source code, APIs, datasets, libraries, templates, or AI-assisted development used by us have been appropriately acknowledged.

Open-Source / External Resources Used

- **Qwen 2.5** — Local open-source LLM for natural-language understanding and SQL generation.
- **SQLCoder** — Reference/baseline for Text-to-SQL generation.
- **LangGraph** — Agent workflow orchestration.
- **SQLAlchemy** — Database connectivity and SQL execution.
- **FAISS** — Semantic retrieval of relevant schema and business context.
- **nomic-embed-text** — Embeddings for semantic schema retrieval.
- **React / Recharts** — Frontend and data visualization libraries.

AI Tools Used

AI coding assistants such as Claude and Chatgpt used for idea refinement, approach validation, automated test generation.

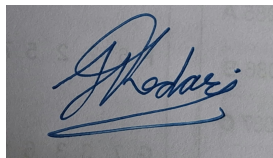
Chatgpt used for grammatical checks, refining language, helping in documentation.

Perplexity used for assisting in research.

SECTION 6 — UPLOADS / ATTACHMENTS CHECKLIST

- ☐ Technical Flow / Architecture Diagram
 - ☐ Research / Reference Document (if applicable)
 - ☐ Project Documentation / Technical Report
 - ☐ GitHub Repository Link
 - ☐ Screenshots of Working Prototype
 - ☐ Dataset / Sample Data (if applicable)
 - ☐ Additional Diagrams (if any)
-

Team Leader Signature:

A handwritten signature in blue ink, appearing to read "J. K. Das", is shown on a piece of paper.

Date: 11 / 9 / 2026